

Group 17

July 10, 2024

Report

Documentation and Reflection on the Practical Sessions

Group Members

Angelina Podolako

Boris Wagemans

Kiran de Wit

Maria Thiele

Course: Introduction to Robotics

Contents

1	Introduction	2
1.1	General approach	2
2	Code documentation and explanation	3
2.1	Line follower	3
2.2	Object Sorter	3
3	Development Process	4
3.1	Week 1-2: Understanding the given materials	4
3.2	Week 3-5: Working on the Line-Follower task	5
3.3	Week 6-8: Working on the Object-Sorter task	5
4	Reflection	6
4.1	Team Work	6
4.2	Development Process	6
4.3	Conclusion	6

1 Introduction

For the practical assignment in the course Introduction to Robotics, a robot was build that can complete two tasks. The first task requires the robot to follow a line. The second task is about sorting objects. While each of these tasks had to be completed by a real robot, the tasks were also implemented in a simulation. This report documents the development process of the robot and the code and concludes with a short reflection on the learning process of the practical assignment.

1.1 General approach

Before we dive into the individual components of the assignment, we would like to shortly introduce ourselves and explain how we generally approached this practical.

Our group includes four students, who are all studying AI in the first year of the Bachelor Program, namely Angelina Podolako, Boris Wagemans, Kiran de Wit and Maria Thiele. In the first workgroup, we divided ourselves into two subgroups. The first subgroup, consisting of Boris and Kiran, would mainly focus on implementing the line follower and object sorter in the simulation, while the second subgroup, consisting of Angelina and Maria, focuses primarily on building the actual robot and implementing both tasks for this robot. The reason for splitting into smaller groups was based on interest and time efficiency. Building the robot did not require the help from all four group members.

Instead, it was more efficient to split the tasks in such a way that two people could focus on building the robot and implementing the code for the actual robot, while the other two members simultaneously focused on getting the simulation to work. For both, the simulation and the actual robot, the line follower task was implemented first. Since we first had to build a robot, parts of the simulation code were already completed when we started to implement the code for the actual robot. This allowed some code copying from the simulation, at least to some extent. The following sections of this report will give a more in-depth explanation to how our code works, what our development process looks like, and a reflection on the practical assignment in general.

2 Code documentation and explanation

Although the code for the simulation and the real robot is similar, there are also some drastic differences. To understand the general idea behind each code, this section focuses on documenting and explaining the code. In addition, the main differences will be highlighted. While this document will not contain specific code fragments, the general idea of each code will be analysed and explained.

2.1 Line follower

The first task was to implement a robot that could follow a line. In this task there is no strong differentiation between the simulation and the real robot.

We implemented a PID controller. The error signal that we get is the light reflection. The robot should always stay on as much white area as black area. The reflection that is detected is used as the basis for the PID controller. While the constants vary in the simulation and the real code, the structure is the same. One function determines a correction that is proportional to the error (P). Another function calculated the rate of change with which the error is changing (D) and the last function takes all of the previous errors into account (I). The sum of the output of all these functions determines the amount by which the error is corrected by.

In an early version, the code for the simulation also included a function that enabled the robot to find the line in case it is not placed on the line. Since this was not required and would have not worked with the real robot, this function is no longer used.

2.2 Object Sorter

For the object sorting task we implemented a subsumption architecture.

The basic functionality of the robot is moving around on the field. The actual robot should not leave the field, but is allowed to cross the middle line. In the simulation, the robot is not allowed to hit a wall.

When a block is detected, the robot drives towards the block and determines the color of the block. Based on the color of the block, the robot performs a task. In the simulation, the robot can either directly sort the block or first compact it. Since the task for the actual robot is simplified, it only determines if the block is located in the correct field or if it needs to be pushed to the opposite field.

While the basic structure of the code is similar for both the simulation and actual robot, the individual implementation varies. This is mainly due to the difference in sensors and task complexity.

3 Development Process

To provide a better understanding on how we structured and divided the tasks, which problems we encountered, and how we solved these problems, we documented the process of development. There are three basic stages of development. First, we familiarized ourselves with the materials given. Second, the line follower task was solved for both the simulation and the real robot. Lastly, we worked on the object sorting task.

3.1 Week 1-2: Understanding the given materials

We started the assignment by first discussing the best strategy for solving all subtasks. Since there were two main tasks (the line follower and the object sorter) that we had to solve in CoppeliaSim and for the real robot, we believed that splitting ourselves into two groups would be the most beneficial. This way, we were able to work on the simulation and the robot at the same time.

To better organize the workgroups and plan our time more efficiently, an excel file was created that documented what we had done in each week. In addition, the excel file also included which tasks had been completed. Moreover, it provided an overview on everyone's individual tasks and deadlines.

Before we started the actual tasks, we familiarized ourselves with the given materials. Since Boris and Kiran planned to work on the simulation, it was necessary to install CoppeliaSim. This proved to be more difficult than anticipated because it did not run on MacOS. At the same time, Angelina and Maria focused on building the real robot. First, the Lego Set was inspected in order to get a rough idea on possible designs for the robot. Next, a first version of the robot was built. Due to the advice of a TA, the initial idea was to build two robots, one for the line following and one for the object sorting. Due to this, the first design of the robot only included the sensors necessary for the line following. Before the line following was implemented, a few simple programs were used to understand the sensors and steering options for the robot.

3.2 Week 3-5: Working on the Line-Follower task

After about two weeks of getting to know the given materials, the line following task was implemented both in the simulation, as well as for the real robot.

Due to CoppeliaSim not running on MacOS, the line following task took longer than expected. Up until the fifth workgroup, only one of our group was effectively able to work on the simulation. In turn, this hindered the development. Because working on the simulation could also be done outside of the workgroup times, this issue was not too severe. For the simulation, we had originally also implemented a way for the robot to find the line in case it is not placed on the line. This took a lot of time and, as we later found out, was not necessary. Since we wanted to reuse the simulation code for the robot, we deleted this part of the code.

While working on the code for the real robot, we found out that it might be beneficial to only build one robot. Since we were initially advised to build two robots, we had to decide between the original idea of having two robots or modifying our line following robot to also be able to solve the object sorting task. While we opposed the idea of modifying our robot due to the limited time, we also did not want to have to switch the robots during the demo day. Therefore, we decided to modify our robot. In the end, we completely build a new robot that is able to switch quickly between both tasks. The incorporation of the sensors proved to be harder than initially believed. While this delayed the development of the robot, the actual code implementation was quicker than expected. The code for the actual robot is very similar to the code for the simulation, hence, the general idea could just be copied. The parameters were adjusted and the line following also worked for the real robot. We did, however, notice that our robot had troubles with the ramp. We ended up making small adjustments. Even though we did not change the entire design of the robot, these adjustments required us to also change the parameters in the code. In the end, we managed to get the line following to work, but it did cost a lot of time.

3.3 Week 6-8: Working on the Object-Sorter task

In the last phase, we focused on implementing the object sorting for both the actual robot and the simulation.

Unlike the line following task, the object sorting task varied a lot between simulation and actual implementation. While the simulation included a charging station and different operations for the cubes, the task was simplified for the actual robot. Because the task in the simulation was more complex, the code for the robot was written independently from the code for the simulation. This was mainly done due to the differences in the tasks but also because of the limited time we had to work on the robot.

Due to the lack of time, we wrote the code for the robot at home and used the workgroups to test the code. While this approached saved us time in the workgroup, we still struggled to get everything to work. Our main problems included the size of the cubes and the uneven mats. To keep track on which side of the field the robot is, we used the gyro sensor. Since the mats were very uneven and wavy, the accuracy of the sensor was

heavily impacted. This caused extreme problems with the basic behavior of the robot. Furthermore, the cubes that should be sorted are very small. Often times, the distance sensor does not detect a cube, even if it is right in front of the robot. This complicated the search for other problems with the code. In the end, we did not manage to complete this task, also because of the lack of time.

Difficulties with this task arose in the simulation as well. Due to the higher complexity of the task, it was more difficult to implement all of the necessary functions. The main difficulties was getting the sensors to work in unison. While we had a basic structure, combining the color detection with finding the blocks proved to be too difficult.

4 Reflection

4.1 Team Work

Looking back on the past weeks, we believe that we worked well as a team. We had a clear structure and organization and splitting into subgroups was most time efficient. Nonetheless, we could have communicated more between the subgroups. Since we worked on rather different aspects every week, we did not take a lot of time to share the process between the groups. For a majority of the time we worked independently. On the one hand, it was not always necessary to know every detail that the other group is working on, on the other hand, more communication could have helped to make some workgroups more efficiently.

4.2 Development Process

We have faced a variety of difficulties throughout the weeks like the difficulties with CoppeliaSim or finding a functional robot design. Despite many problems, we did manage to get the line following to work both in the simulation and with the real robot. However, the object sorting works in neither of the two. In the end, the lack of time has prohibited us from completing the object sorting. We should have planned more time on this task. For example, in the first weeks, we were not very productive as we were trying to get familiar with the materials. By limiting the time we spent on experimenting, we could have had more time to spend on the object sorting.

4.3 Conclusion

During this practical assignment we learned to build a robot and to run code in a simulation. While the simulation is a nice way of trying out various ideas, it is not always feasible to use the simulation code for an actual robot. Problems that occur with the simulation are not necessarily occurring when programming the real robot and vice versa. The simulation enables for a better control over specific functions but the

real robot is limited by the available sensors. We believe that while it is useful to use the simulation for visualizing a specific task, the real implementation of such a task can vary drastically. Therefore, it might not always be necessary to implement a simulation. Overall, we enjoyed getting to know both ways of implementing a robot and we wish that we had more time.